FastGA

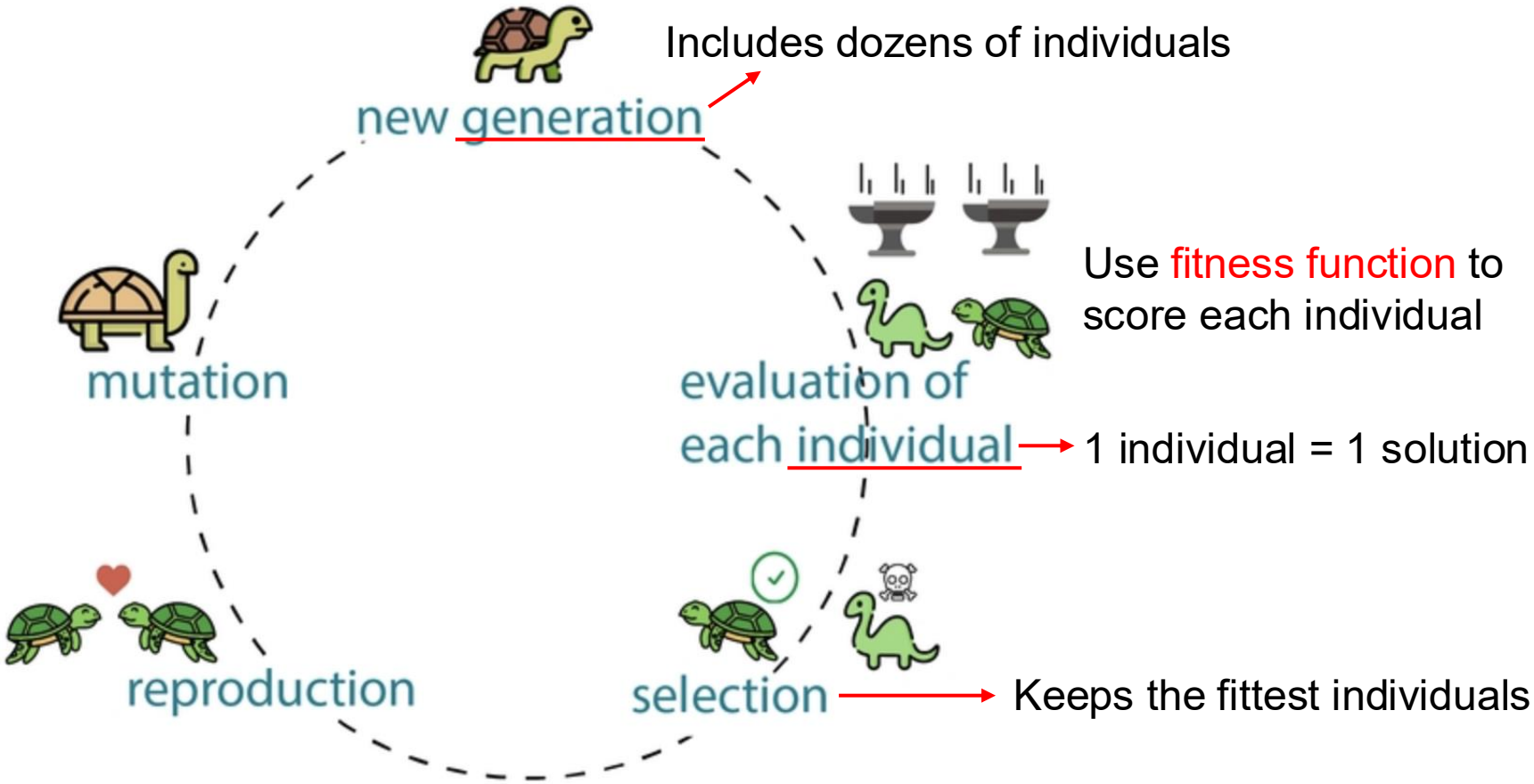
Numerical Software Development Final Presentation

Jim Lin (@jylin34)

June 8, 2026 | 08:15 – 08:30

Genetic Algorithm (GA)

- Genetic Algorithm is a heuristic algorithm inspired by Darwin natural selection
- Solve complex optimization problems by involving a population of solutions




```
Input: fitness_function()
for each generation:
    for each individual:
        evaluate(individual)
    selection()
    reproduction()
    mutation()
Output: Fittest individual
```

FastGA

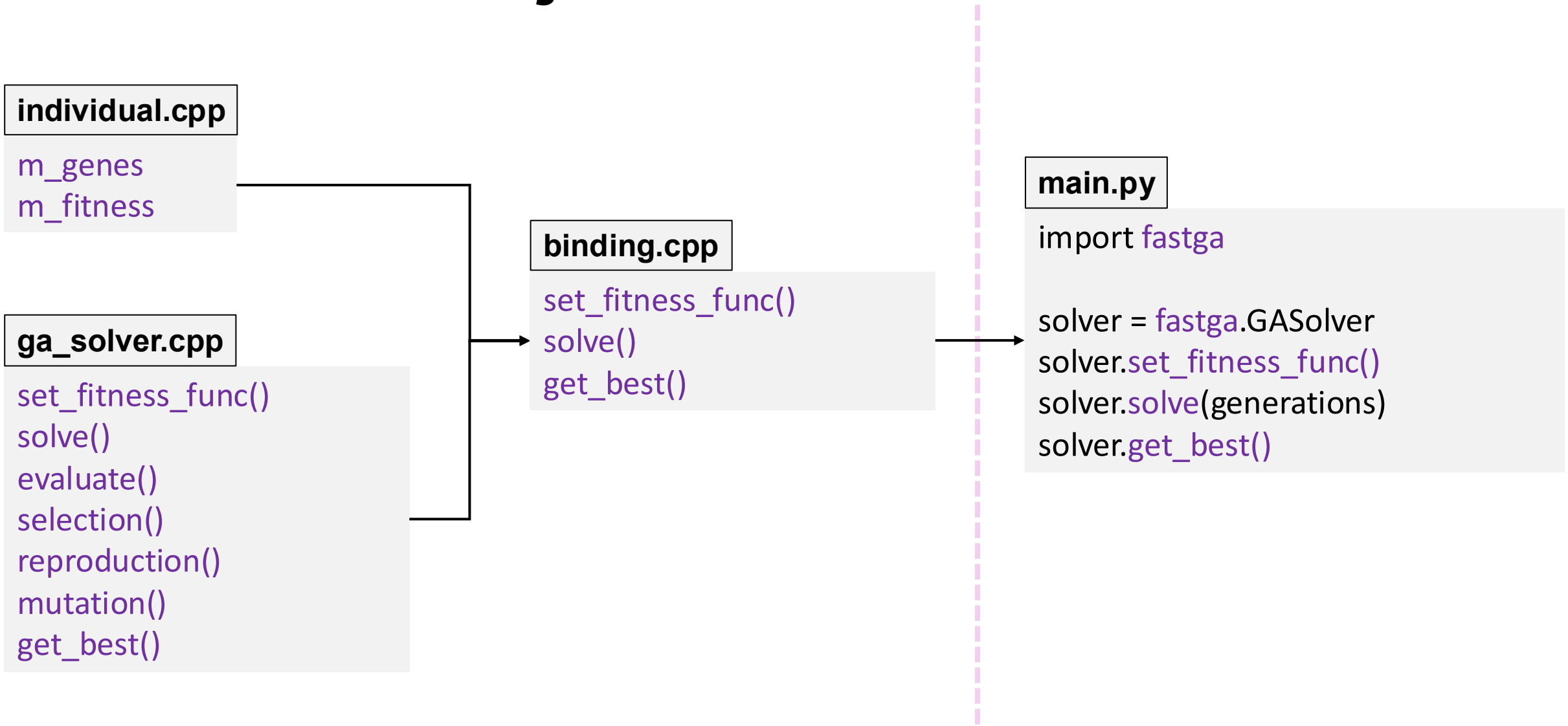
- FastGA is a C++ genetic algorithm library with Python bindings, specifically for global optimization problems represented by 1D double-precision vectors
- eg.
  - fitness function: $\min_{(X,Y)} X^2 + Y^2$
  - Individual: $[X, Y]$
  - Goal: fitness value = 0, $[\mathrm{X}, \mathrm{Y}] = [0, 0]$

Thousands of generations amplify the minor interpreter overhead into significand delays

Calling the Python fitness function for every single individual incurs massive overhead from C++-to-Python context switching and GIL contention during every evaluation

```
Input: fitness_function()
for each generation:
    for each individual:
        evaluate(individual)
    selection()
    reproduction()
    mutation()
Output: Fittest individual
```

Project Architecture



set_fitness_func() & evaluate()

main.py

```
solver.set_fitness_func(x)
```

Stores the python fitness function as a callable reference in C++

ga_solver.cpp

```
void set_fitness_func(py::function func){
    m_fitness_func = func;
}
```

ga_solver.cpp

```
void GASolver::evaluate(){
    for each individual:
        py::array_t<double> genes();
        result = m_fitness_func(genes);
}
```

Wrap C++ memory into a Numpy array, passing it to Python via zero-copy

Can we make it faster?

evaluate() & batch_evaluate()

evaluate()

```
for each individual:
    py::array_t<double> genes();
    result = m_fitness_func(genes);
```

genes = $[x_1, y_1, z_1 \ \ldots\ldots]$ → fitness_func()

genes = $[x_2, y_2, z_2 \ \ldots\ldots]$ → fitness_func()

genes = $[x_3, y_3, z_3 \ \ldots\ldots]$ → fitness_func()

.
.
.

batch_evaluate()

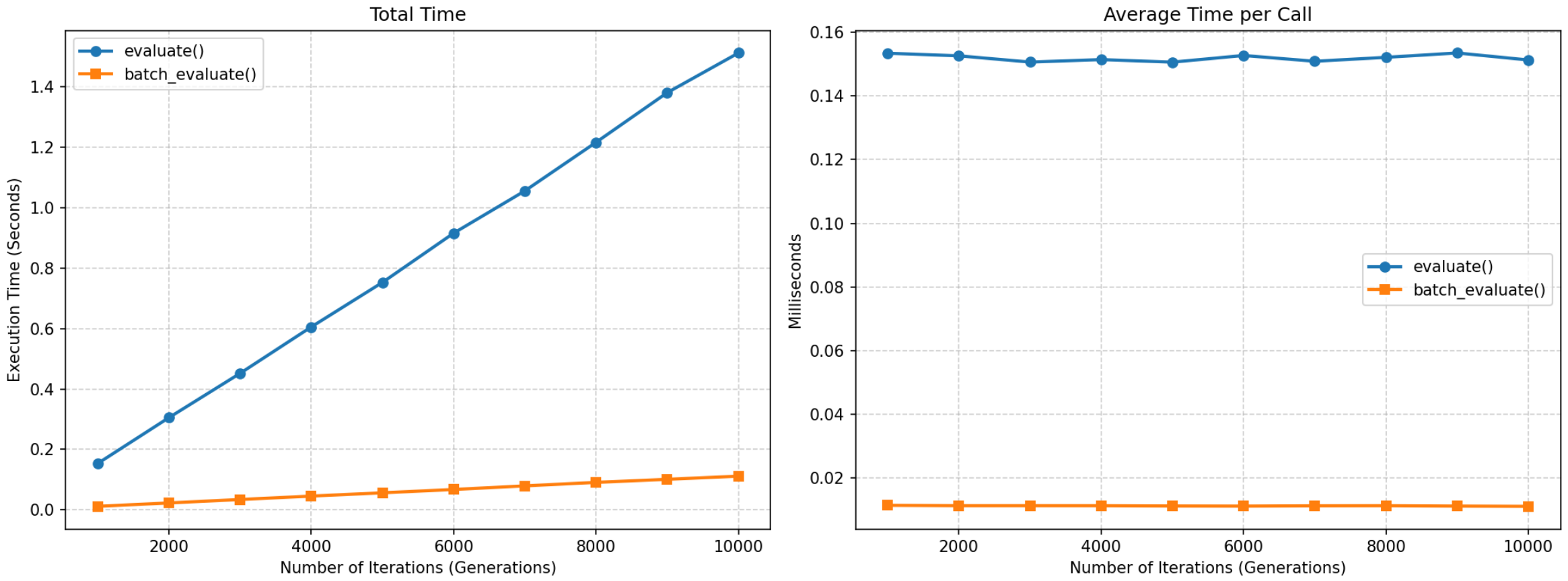
```
py::array_t<double> pop_matrix({
    pop_size, genome_size
});
for each individual:
    pop_matrix[i] = ind[i].genes();
result = m_fitness_func(pop_matrix);
```

pop_matrix[] = $[x_1, y_1, z_1 \ \ldots\ldots]$
$[x_2, y_2, z_2 \ \ldots\ldots]$
$[x_3, y_3, z_3 \ \ldots\ldots]$ → fitness_func()

Pros: fast

Cons: requires users write batch-compatible fitness function

evaluate() vs. batch_evaluate()

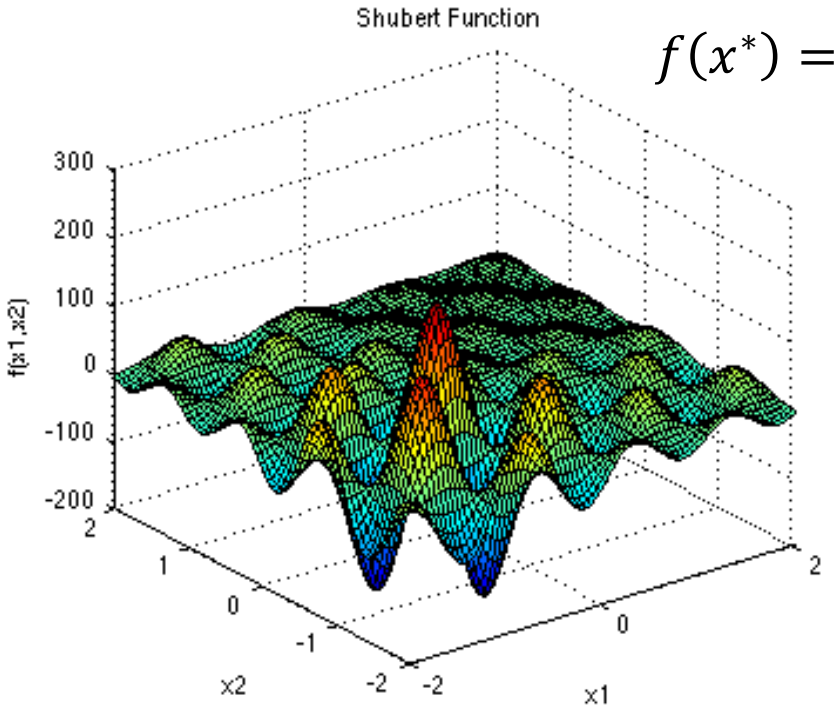


Use Case Example

Shubert Function

genes = $[x, y]$

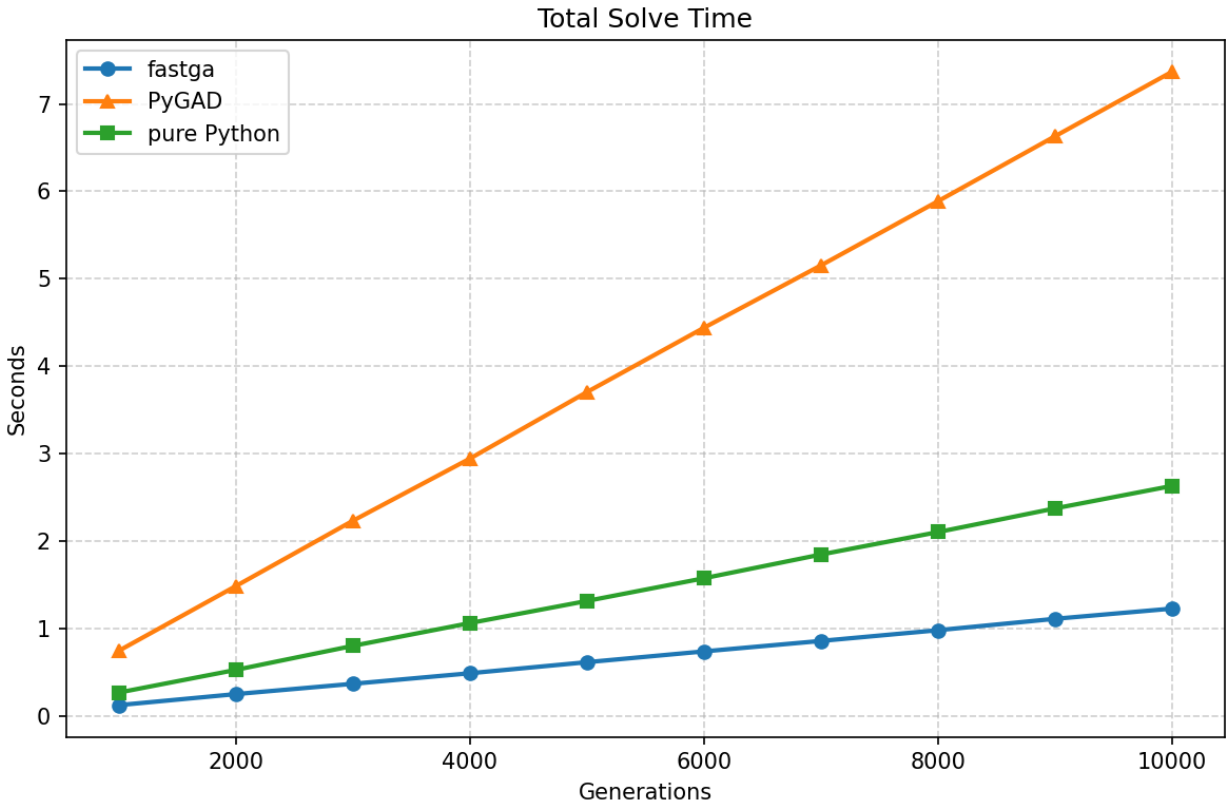
$-2 \leq x, y \leq 2$

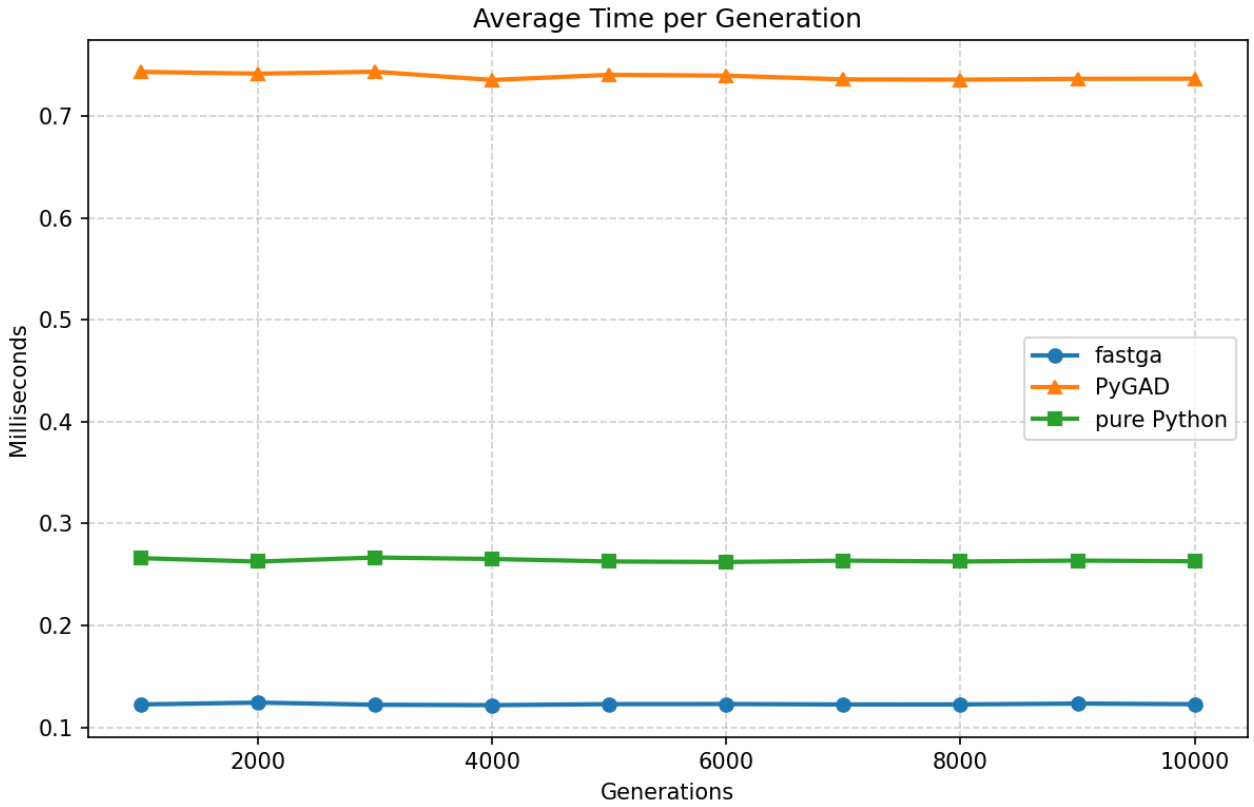


$f(x^*) = -186.7309$

```
Gen 979 best fitness=-186.731 genes=[-1.42505, -0.800262]
Gen 980 best fitness=-186.731 genes=[-1.42505, -0.800262]
Gen 981 best fitness=-186.731 genes=[-1.42505, -0.800262]
Gen 982 best fitness=-186.731 genes=[-1.42505, -0.800262]
Gen 983 best fitness=-186.731 genes=[-1.42505, -0.800262]
Gen 984 best fitness=-186.731 genes=[-1.42505, -0.800262]
Gen 985 best fitness=-186.731 genes=[-1.42505, -0.800262]
Gen 986 best fitness=-186.731 genes=[-1.42505, -0.800262]
Gen 987 best fitness=-186.731 genes=[-1.42505, -0.800262]
Gen 988 best fitness=-186.731 genes=[-1.42505, -0.800262]
Gen 989 best fitness=-186.731 genes=[-1.42505, -0.800262]
Gen 990 best fitness=-186.731 genes=[-1.42505, -0.800262]
Gen 991 best fitness=-186.731 genes=[-1.42505, -0.800262]
Gen 992 best fitness=-186.731 genes=[-1.42505, -0.800262]
Gen 993 best fitness=-186.731 genes=[-1.42505, -0.800262]
Gen 994 best fitness=-186.731 genes=[-1.42505, -0.800262]
Gen 995 best fitness=-186.731 genes=[-1.42505, -0.800262]
Gen 996 best fitness=-186.731 genes=[-1.42505, -0.800262]
Gen 997 best fitness=-186.731 genes=[-1.42505, -0.800262]
Gen 998 best fitness=-186.731 genes=[-1.42505, -0.800262]
Gen 999 best fitness=-186.731 genes=[-1.42505, -0.800262]
Best individual genes: [-1.4250505292262396, -0.8002623490361939]
Best individual fitness: -186.73088705981402
 ~/Doc/m/114_2/N/FastGA/python  develop !3
```

FastGA vs. PyGAD vs. pure Python





References

- What is a Genetic Algorithm? https://www.generativedesign.org/02-deeper-dive/02-04_genetic-algorithms/02-04-01_what-is-a-genetic-algorithm
- Genetic Algorithm Wikipedia https://en.wikipedia.org/wiki/Genetic_algorithm
- Shubert Function https://www.sfu.ca/~ssurjano/shubert.html
- PyGAD Github https://github.com/ahmedfgad/GeneticAlgorithmPython